File permissions in Linux

Project description

Project: File System Permissions Audit and Authorization Management

Scenario Overview
As a security professional supporting a large organization's research team, I audited and updated file system permissions to ensure they matched required authorizations, preventing unauthorized access and upholding security.

Key Responsibilities

- Used `ls -la` to inspect permissions on files and directories (including hidden ones) in the researcher's projects folder, identifying mismatches.
- Applied `chmod` (e.g., `chmod u-w,g-rx`) to adjust read, write, and execute permissions for users, groups, and others per least-privilege rules.
- Verified and modified ownership/group access on directories like drafts to restrict to authorized users (e.g., researcher2 only).
- Captured before/after screenshots and command outputs for documentation.

Tools and Commands Used

- `ls -la`: Detailed listing of permissions, owners, and groups.
- `chmod`: Modified permissions (e.g., remove group execute on drafts: `chmod g-x drafts`).
- Ownership checks/adjustments via lab context (e.g., ensuring researcher2 access).

Outcomes

- Permissions aligned with policies, enforcing confidentiality and least privilege.
- Reduced risks from over-permissions, with full audit trail provided.

Skills Demonstrated
Linux permissions auditing, `chmod` for access control, authorization enforcement, risk assessment.

Check file and directory details

In the `/home/researcher2/projects` directory, there are five files with the following names and permissions:

- `project_k.txt` -rw-rw-rw-
  - User = read, write,
  - Group = read, write
  - Other = read, write
- `project_m.txt` -rw-r-----
  - User = read, write
  - Group = read
  - Other = none
- `project_r.txt` -rw-rw-r--
  - User= read, write
  - Group = read, write
  - Other = read
- `project_t.txt` -rw-rw-r--
  - User = read, write
  - Group = read, write
  - Other = read
- `.project_x.txt` -rw--w----
  - User = read, write
  - Group = write
  - Other = none

There is also one subdirectory inside the `projects` directory named drafts. The permissions on `drafts` are: drwx--x---

- User = read, write, execute
- Group = execute
- Other = none

```
researcher2@e8c9cdb22dc3:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Feb  5 08:49 .
drwxr-xr-x 3 researcher2 research_team 4096 Feb  5 09:47 ..
-rw--w---- 1 researcher2 research_team   46 Feb  5 08:49 .project_x.
txt
drwx--x--- 2 researcher2 research_team 4096 Feb  5 08:49 drafts
-rw-rw-rw- 1 researcher2 research_team   46 Feb  5 08:49 project_k.t
xt
-rw-r----- 1 researcher2 research_team   46 Feb  5 08:49 project_m.t
xt
-rw-rw-r-- 1 researcher2 research_team   46 Feb  5 08:49 project_r.t
xt
-rw-rw-r-- 1 researcher2 research_team   46 Feb  5 08:49 project_t.t
xt
```

Describe the permissions string

Linux permissions strings like `drwxr-xr-x` consist of 10 characters: type + 9 permission bits (user/group/other, each with rwx).

- d: Directory (not a regular file).
- rwx (owner/user): Owner has full read (r), write (w), and execute (x) access—can list, modify, and enter the directory.
- r-x (group): Group has read (r) and execute (x), but no write—can list contents and enter, but not modify.
- r-x (others): Everyone else has read and execute, but no write—same limited access.
- A hyphen (-) as the first character in a Linux permissions string (e.g., `-rwxr-xr-x`) indicates it's a regular file, unlike `d` which marks directories.

Context in Lab

This matches the "before" state in the Google Cybersecurity Manage Authorization lab: researcher2 (group "research_team") has read/execute access to project files/drafts, but the task requires tightening (e.g., via `chmod g-x drafts` to revoke group execute). All items owned by researcher2:research_team, dated Feb 5, 10:49

Change file permissions

Objective: Fix specific files violating policy—no write (w) for "others" anywhere; `project_m.txt` owner-only (no group/other read/write).

Step 1: Remove Other Write from project_k.txt

- Run `ls -l` to scan: Identifies `project_k.txt` with "other" w (e.g., `-rwxrwxr-w` where last group is `r-w`).
- Why Vulnerable: Others (non-group users) can modify/delete, breaching confidentiality/integrity.
- Fix: `chmod o-w project_k.txt` (revokes write specifically for others). Result: Last triplet becomes `r--` or `---`.

Step 2: Restrict project_m.txt to Owner-Only

- Run `ls -l`: Group has read (r-- in middle triplet).

- Policy Need: Restricted file—only owner (researcher2) gets rw; group/other get nothing.
- Fix: `chmod g-r project_m.txt` (removes group read; write already absent). Result: Middle triplet `---`.

chmod Syntax Deep Dive:

- Targets: `u` (user/owner), `g` (group), `o` (others), `a` (all).
- Operators: `+` (add), `-` (remove), `=` (exact set).
- Permissions: `r` (read contents/list), `w` (modify/add/delete), `x` (execute/enter dir).
- Example: `chmod o-w,g-r file` = precise revocation without affecting owner.

Verification: Rerun `ls -l` post-changes; "Check my progress" confirms via lab script. Outcomes align CIA triad: Confidentiality (no unauthorized reads), Integrity (no writes)

Change file permissions on a hidden file

This task targets `.project_x.txt` (hidden due to leading `.`)—an archived file where user/group should read (r) but not write (w), enforcing read-only archival security via precise `chmod`.

Task Details

Objective: Revoke write access for both user and group while preserving read; "others" already restricted.

Step 1: Inspect with `ls -la`

- Reveals hidden `.project_x.txt` permissions (e.g., `-rw-r--r--` or similar with `rw` for owner/group).
- Issue Identified: User/group have `w` (write)—violates "no writing to archived file" policy.

Step 2: Apply `chmod u-w,g-w,g+r .project_x.txt`

- `u-w`: Removes write from user/owner (keeps r,x if present).
- `g-w`: Removes write from group (keeps r if present).
- `g+r`: Ensures group gets read (if somehow missing).
- Result: Permissions become `-r--r--r--` or equivalent (read-only for user/group; others unchanged).

Why This Command?

- Symbolic mode (`u,g,o`) targets specifics without resetting everything (unlike numeric 644).
- Hidden files need `ls -la` (not plain `ls`); dot (`.`) prefix is key.
- Aligns least privilege: Read access maintained for authorized parties, writes blocked everywhere.

Verification: Rerun `ls -la .project_x.txt`—confirm no `w` in owner/group triplets. Strengthens CIA (Confidentiality/Integrity via no-modification).

Change directory permissions

This task secures the `drafts` subdirectory by revoking group execute (x) access, limiting traversal/contents viewing to owner `researcher2` only—critical for sensitive draft isolation.

Task Breakdown

Objective: From `projects` dir, restrict `drafts` to owner-only access (execute needed to enter/list dirs).

Step 1: Inspect with `ls -l`

- Shows `drafts` as `drwxr-xr-x` (or similar): owner rwx; group r-x (read + execute allows group to `cd drafts` and `ls` contents).
- Issue: Group "research_team" has `x`, enabling unauthorized browsing—violates "only researcher2" policy.

Step 2: Fix with `chmod g-x drafts`

- `g-x`: Specifically strips execute from group (middle triplet loses `x`, e.g., becomes `drwxr----`).
- Effects:
  - Owner retains rwx (full access).
  - Group drops to rw- or r-- (can't enter without x).
  - Others unchanged (likely none).
- Directory `x` means "traverse/search" (cd/ls); without it, group can't access contents even if read present.

Key Concepts

- Dirs need `x` for entry (like a locked door); `r` lists contents once inside.
- Command run from parent (`projects`) targets subdir directly.
- Post-fix: `ls -l` confirms `drwxr----`—owner-only enforcement.

Security Impact: Prevents lateral movement by group members, upholding least privilege for drafts

Summary

This Google Cybersecurity lab demonstrates auditing and securing research team files via Linux permissions, enforcing least privilege across files, hidden files, and directories.

Project Description

File System Permissions Audit and Authorization Management
As a security professional, I audited `/home/researcher2/projects` permissions using `ls -la`, identified over-permissions (others write on `project_k.txt`, group read on `project_m.txt`, etc.), and applied targeted `chmod` fixes to align with authorization policies—preventing unauthorized access while maintaining CIA triad compliance.

Initial Permissions State

File/Dir	Permissions	Issues Identified
`project_k.txt`	`-rw-rw-rw-`	**Others: rwx** (full access)
`project_m.txt`	`-rw-r-----`	**Group: r** (unauthorized read)
`project_r.txt`	`-rw-rw-r--`	Group write
`project_t.txt`	`-rw-rw-r--`	Group write
`.project_x.txt`	`-rw--w----`	**User/group: w** (archived file)
`drafts/`	`drwx--x---`	**Group: x** (can traverse)

Task-by-Task Fixes

Task 1: Baseline Audit (`ls -la`)
Revealed uniform `drwxr-xr-x`/`-rw-r--r--` pattern—group/other had excessive r-x access across projects/drafts.

Task 2: File Permissions

- `project_k.txt`: `chmod o-w` → removes others write (`-rw-rw-r--`)
- `project_m.txt`: `chmod g-r` → owner-only (`-rw-------`)

Task 3: Hidden File (`.project_x.txt`)
`chmod u-w,g-w,g+r` → read-only archive (`-r--r-----`) for user/group.

Task 4: Directory (`drafts`)
`chmod g-x drafts` → owner-only traversal (`drwxr-----`) blocks group `cd/ls`.

Key Skills Demonstrated

- Permissions parsing: `drwxr-xr-x` → type + u/g/o triplets (rwx hierarchy)
- Symbolic `chmod`: `u/g/o` + `-rwx` for precise revocation
- Hidden files: `.name` + `ls -la` detection
- Directory semantics: `x`=traverse (entry required for `r`=list)

Outcomes: Full audit trail with before/after screenshots; reduced attack surface by eliminating group/other over-permissions. Portfolio-ready for GRC roles emphasizing access control.